



1. 最小生成树

概念

Prim算法

Kruskal算法

1. 最小生成树

概念

Prim算法

Kruskal算法

BOK数据结构



生成树：包含图中所有顶点的一个极小连通子图

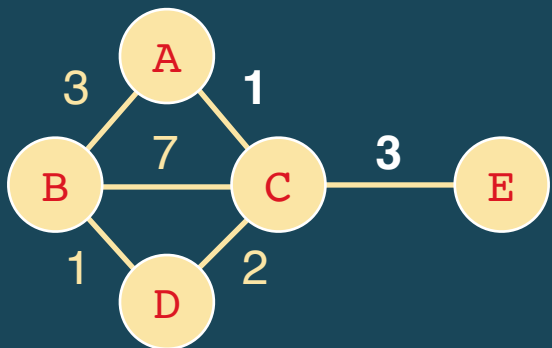
最小生成树：各边代价之和最小的那棵生成树

1. 生成的边的数量一定是 $n-1$

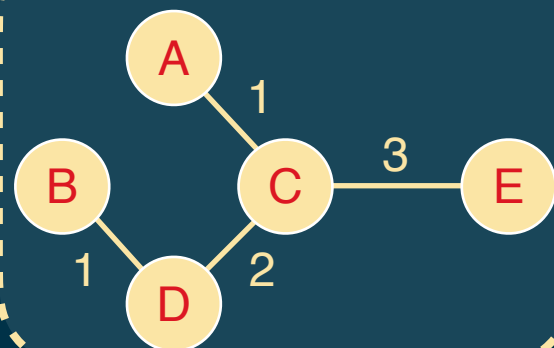
2. 若原图存在权值相同的边，那么该图的最小生成树可能不唯一

唯一生成树

带权连通无向图

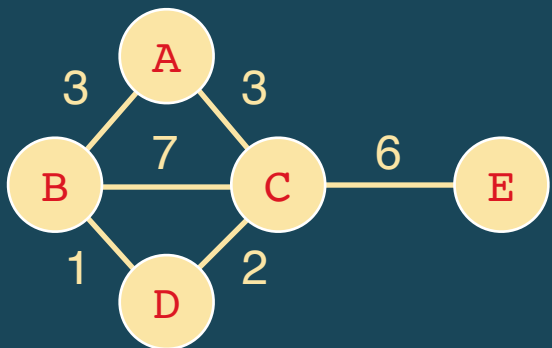


最小生成树

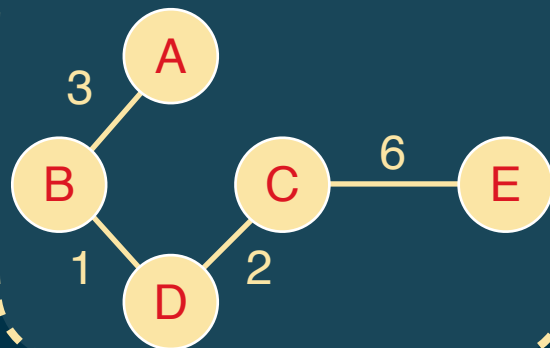


不唯一生成树

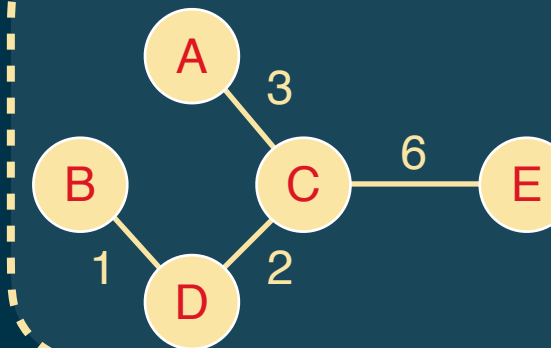
带权连通无向图



最小生成树1



最小生成树2





1. 最小生成树

概念

Prim算法

Kruskal算法

生成树：包含图中所有顶点的一个极小连通子图

最小生成树：各边代价之和最小的那棵生成树

1. 生成的边的数量一定是 $n-1$
2. 若原图存在权值相同的边，那么该图的最小生成树可能不唯一
3. 若原图的边权值互不相等。那么该图的最小生成树唯一
4. 最小生成树中所有边的权值之和是所有生成树中最小的



1. 最小生成树

概念

Prim算法

Kruskal算法

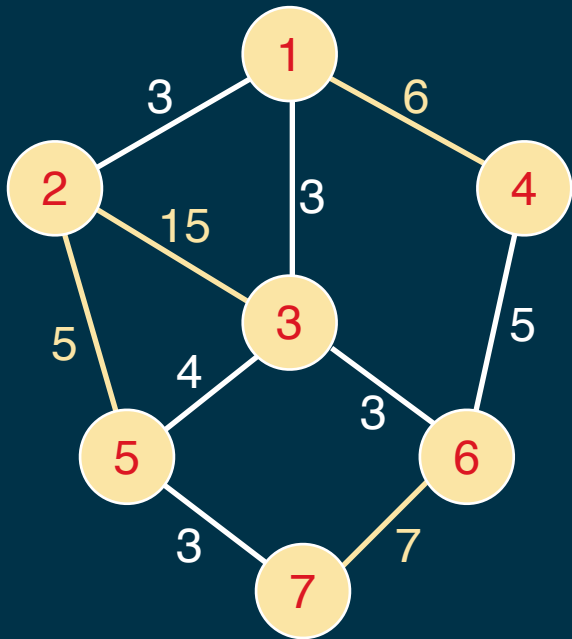
算法流程

1. 初始时任取一顶点加入树T
2. 选择一个与当前T中顶点集合距离最近的顶点, 并将其顶点和相应边加入T, 每次操作后T的顶点数和边数都加一

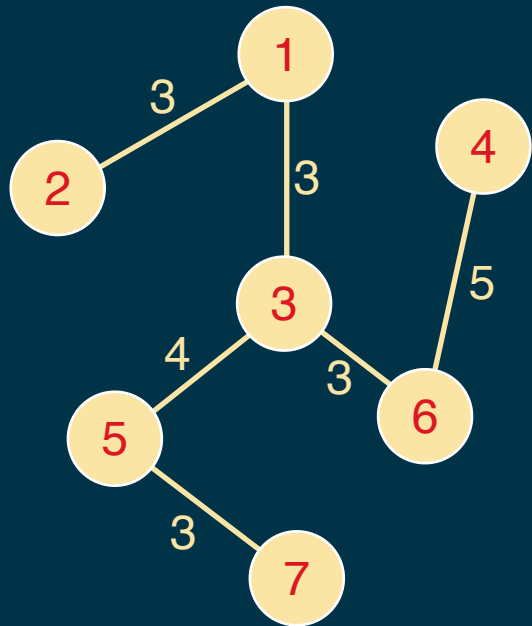
3. 直到所有顶点加入T, 此时有n-1条边

逻辑示意

MST (最小生成树) 代价:
 $3+3+3+3+4+5=21$



图



对应最小生成树



1. 最小生成树

概念

Prim算法

Kruskal算法

算法流程

1. 初始时任取一顶点加入树 T
2. 选择一个与当前 T 中顶点集合距离最近的顶点, 并将其顶点和相应边加入 T , 每次操作后 T 的顶点数和边数都加一
3. 直到所有顶点加入 T , 此时有 $n-1$ 条边

时间复杂度: $O(|V|^2)$
不依赖于 E (边数)

1. 最小生成树

概念

Prim算法

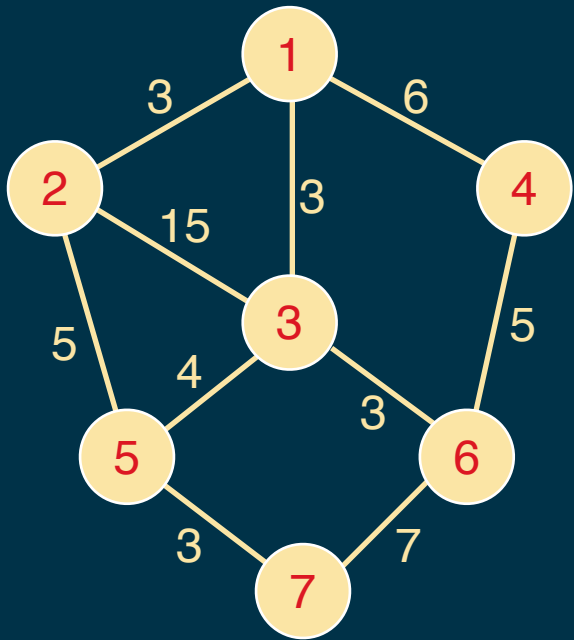
Kruskal算法
(适用于边稀疏, 顶点多的图)

算法流程

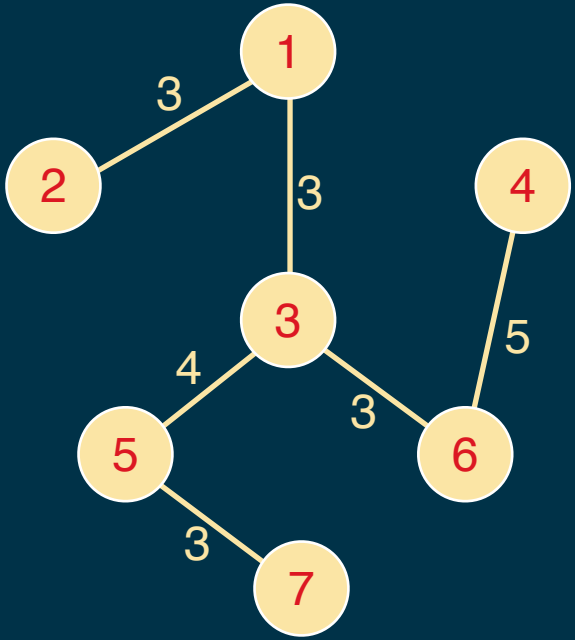
1. 初始时为只有n个顶点而无边的非连通图T, 每个顶点自成一个连通分量
2. 按照边的权值从小到大的顺序, 选择当前未被选取且权值最小的边。若该边依附的顶点落在T中不同的连通分量上, 则将此边加入T, 否则舍弃此边而选择下一条
3. 直到T中所有顶点都在一个连通分量上

逻辑示意

MST (最小生成树) 代价:
 $3+3+3+3+4+5=21$



图



对应最小生成树



1. 最小生成树

概念

Prim算法

Kruskal算法
(适用于边稀疏, 顶点多的图)

算法流程

1. 初始时为只有 n 个顶点而无边的非连通图 T , 每个顶点自成一个连通分量
2. 按照边的权值从小到大的顺序, 选择当前未被选取且权值最小的边。若该边依附的顶点落在 T 中不同的连通分量上, 则将此边加入 T , 否则舍弃此边而选择下一条
3. 直到 T 中所有顶点都在一个连通分量上

时间复杂度: $O(|E| \log_2 |E|)$
算法复杂度与图中的顶点数无关



2. 最短路径

概念

Dijkstra算法

Floyd算法



2. 最短路径

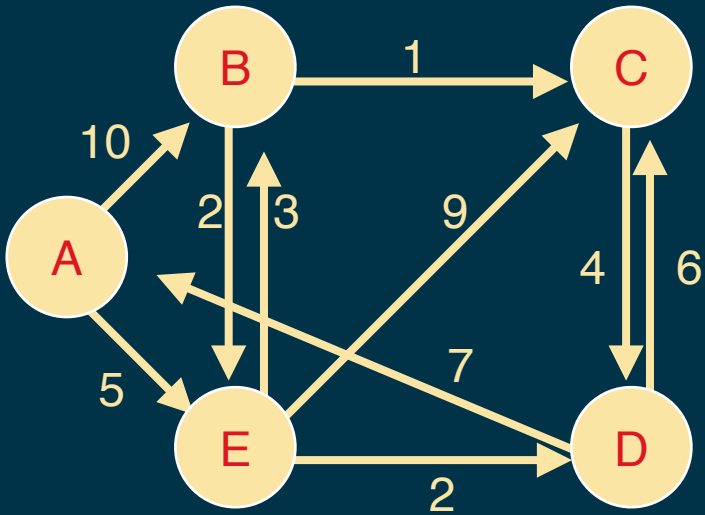
概念

Dijkstra算法

Floyd算法

在图中以最小代价从一个节点到达另一个节点

A如何以最小代价到达其余各点





2. 最短路径

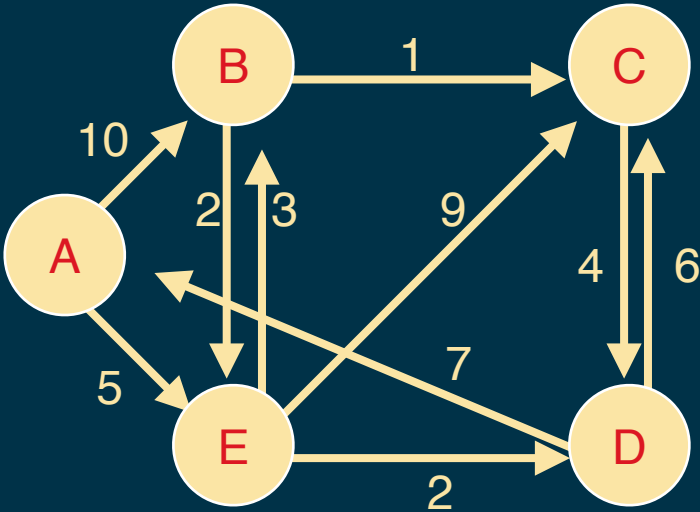
概念

Dijkstra算法

Floyd算法

Dijkstra: 求某个源点到其余各顶点的最短路径

A如何以最小代价到达其余各点



2. 最短路径

概念

Dijkstra算法

Floyd算法

BOK数据结构



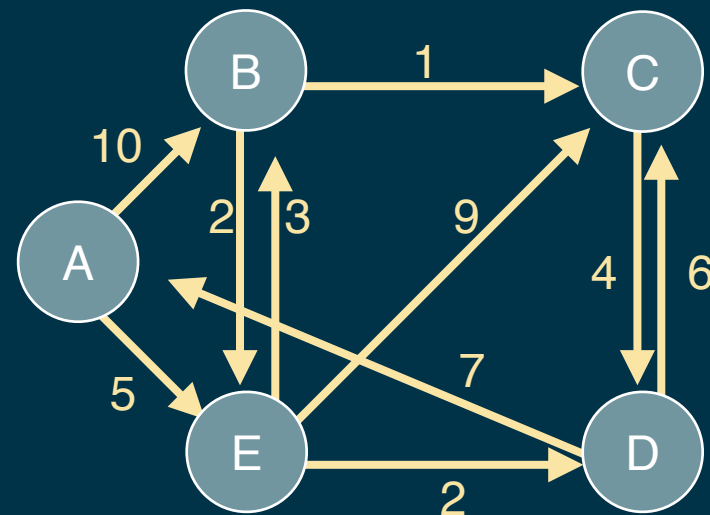
Dijkstra: 求某个源点到其余各顶点的最短路径

算法流程

1. 初始化三个数组信息
2. 观察`final[]`数组, 若`final[i]=0`且`dist[i]`最小, 则将`final[i]`改为1, 收录该顶点`vi`到最优路径集合中
3. 对`vi`的邻接点`vj`做如下判断:
如果`dist[i]+<vi,vj>`的权值`<dist[j]`, 则更新`dist[j]`与`path[j]`

注意: Dijkstra不适用于边上带有负权值的图!

A如何以最小代价到达其余各点



<code>final[5]</code>	1	1	1	1	1	是否已找到最短路径
<code>dist[5]</code>	0	8	9	7	5	最短路径长度
<code>path[5]</code>	-1	4	1	4	0	路径上的前驱
数组下标	A 0	B 1	C 2	D 3	E 4	



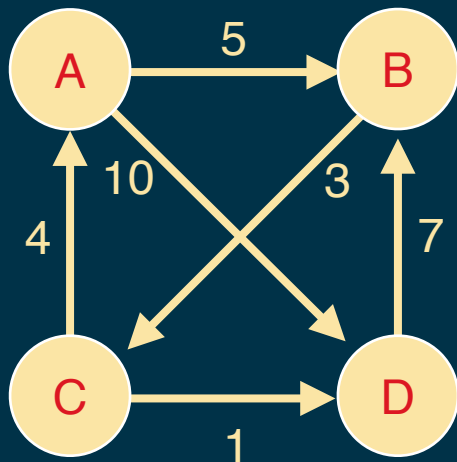
2. 最短路径

概念

Dijkstra算法

Floyd算法

Floyd: 求每一对顶点之间的最短路径



2. 最短路径

概念

Dijkstra算法

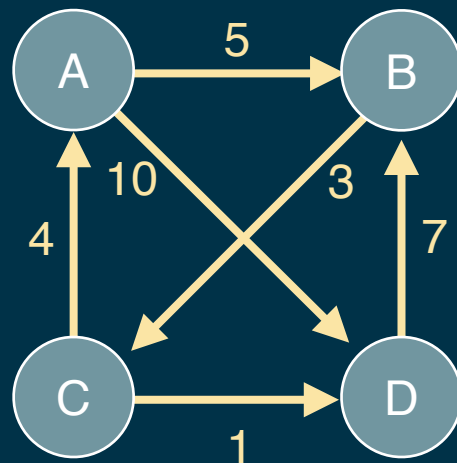
Floyd算法

BOK数据结构



Floyd: 求每一对顶点之间的最短路径

求每一对顶点之间的最短路径



	A	B	C	D
A	0	5	8	9
B	7	0	3	4
C	4	8	0	1
D	14	7	10	0

$A^{(4)}$

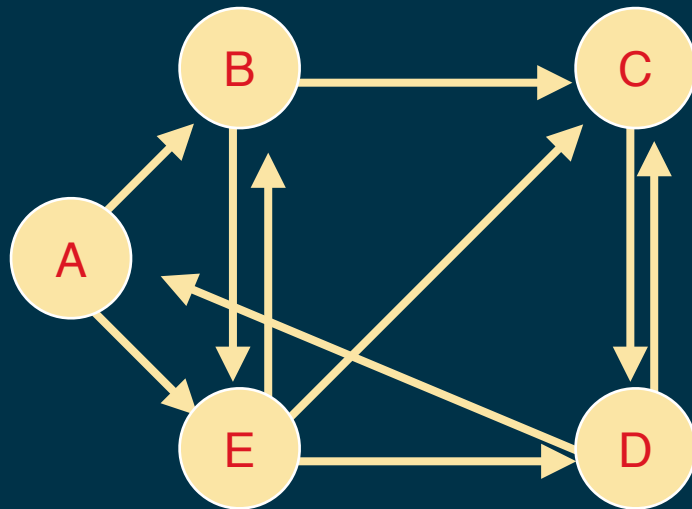
Floyd算法的时间复杂度为 $O(|V|^3)$
允许有负权值的边，但不允许包含带负权值的边组成的回路



2. 最短路径

BFS方法(仅非带权图)

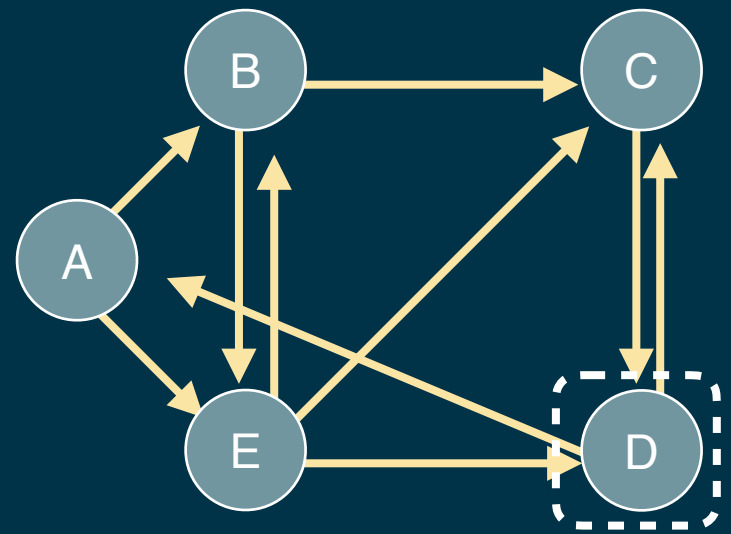
```
bool visited[MVNum];
void BFS_distance(Graph G, int v) {
    int i, u, w;
    int d[G.vexnum];
    for (i = 0; i < G.vexnum; i++) {
        visited[i] = false;
        d[i] = ∞;
    }
    visited[v] = true;
    d[v] = 0;
    InitQueue(Q);
    EnQueue(Q, v);
    while (!QueueEmpty(Q)) {
        DeQueue(Q, u);
        for (w = FirstAdjVex(G, u); w >= 0; w = NextAdjVex(G, u, w)) {
            if (!visited[w]) {
                visited[w] = true;
                d[w] = d[u] + 1;
                EnQueue(Q, w);
            }
        }
    }
}
```



2. 最短路径

BFS方法(仅非带权图)

```
bool visited[MVNum];
void BFS_distance(Graph G, int v) {
    int i, u, w;
    int d[G.vexnum];
    for (i = 0; i < G.vexnum; i++) {
        visited[i] = false;
        d[i] = ∞;
    }
    visited[v] = true;
    d[v] = 0;
    InitQueue(Q);
    EnQueue(Q, v);
    while (!QueueEmpty(Q)) {
        DeQueue(Q, u);
        for (w = FirstAdjVex(G, u); w >= 0; w = NextAdjVex(G, u, w)) {
            if (!visited[w]) {
                visited[w] = true;
                d[w] = d[u] + 1;
                EnQueue(Q, w);
            }
        }
    }
}
```



d[5]	0	1	2	2	1
------	---	---	---	---	---

数组即为所求非带权图中A到各顶点最短路径

visited[5]	T	T	T	T	T
------------	---	---	---	---	---